

Assignment #3 — System Design & Implementation Plan

MTAT.03.229 Enterprise System Integration (Spring 2026)

System: QuickBite — Food Delivery System

Group 7: Alfa-Kilo | Elephant-Yankee | Mike-Alfa | Sierra-Lima

1. Introduction

This document builds on the QuickBite design from Assignment 1 and Assignment 2. It describes a detailed plan for the implementation phase of the project. It covers:

- the full microservices architecture;
- the REST APIs and data models of the services we will build;
- the main end-to-end workflows;
- the synchronous and asynchronous integration mechanisms;
- the token-based security layer used to protect service access;
- the final implementation tasks for each team member.

The business architecture is kept separate from the technical implementation view. Figure 1a stays business-only, while Figure 1b adds the client, gateway, messaging, and database details needed for the implementation plan.

2. Complete System Architecture

2.1 Business Services (from Assignment 1)

QuickBite has **eight business microservices**. We keep all eight in the architecture so that the full business logic is still visible. Only a part of them will be implemented in the project.

#	Service	Main Responsibility	Key Data Owned
1	User Service	Handles sign-up, sign-in, access-token issuance, and user profiles for customers, drivers, restaurant owners, and admins.	Users, roles, addresses
2	Restaurant Service	Handles restaurant sign-up, profile, location, opening hours, and open/closed status.	Restaurants
3	Menu Service	Publishes each restaurant's catalogue: items, prices, and availability.	Menu items
4	Order Service	Owns the order life cycle (Placed → Paid → Confirmed → Prepared → Ready → PickedUp → Delivered) and drives order placement.	Orders, order items
5	Payment Service	Processes payments and refunds for orders (simulated).	Payments, transactions
6	Delivery Service	Creates delivery tasks, matches drivers, and tracks delivery progress.	Delivery tasks
7	Notification Service	Sends messages to users when a status changes.	Notifications
8	Review Service	Collects ratings and reviews after delivery and keeps the average rating per restaurant.	Reviews, rating aggregates

2.2 Service Relationships (business level)

- **Order Service** is the main coordinator. It calls User, Restaurant, Menu, Payment, and Delivery Services to place an order. It also tells Notification Service about status changes.
- **Delivery Service** reads driver profiles from User Service and exchanges status updates with Order Service. It also tells Notification Service about driver assignment and new delivery states.
- **Review Service** checks with Order Service (a review is only allowed after Delivered) and with User Service (to link the review to the reviewer).
- **Menu Service** depends on Restaurant Service, because menus belong to a restaurant.
- **Notification Service** is called asynchronously by any service that needs to inform the user about a status change.
- **User Service** is used by every other service for identity, token issuance, and access checks.

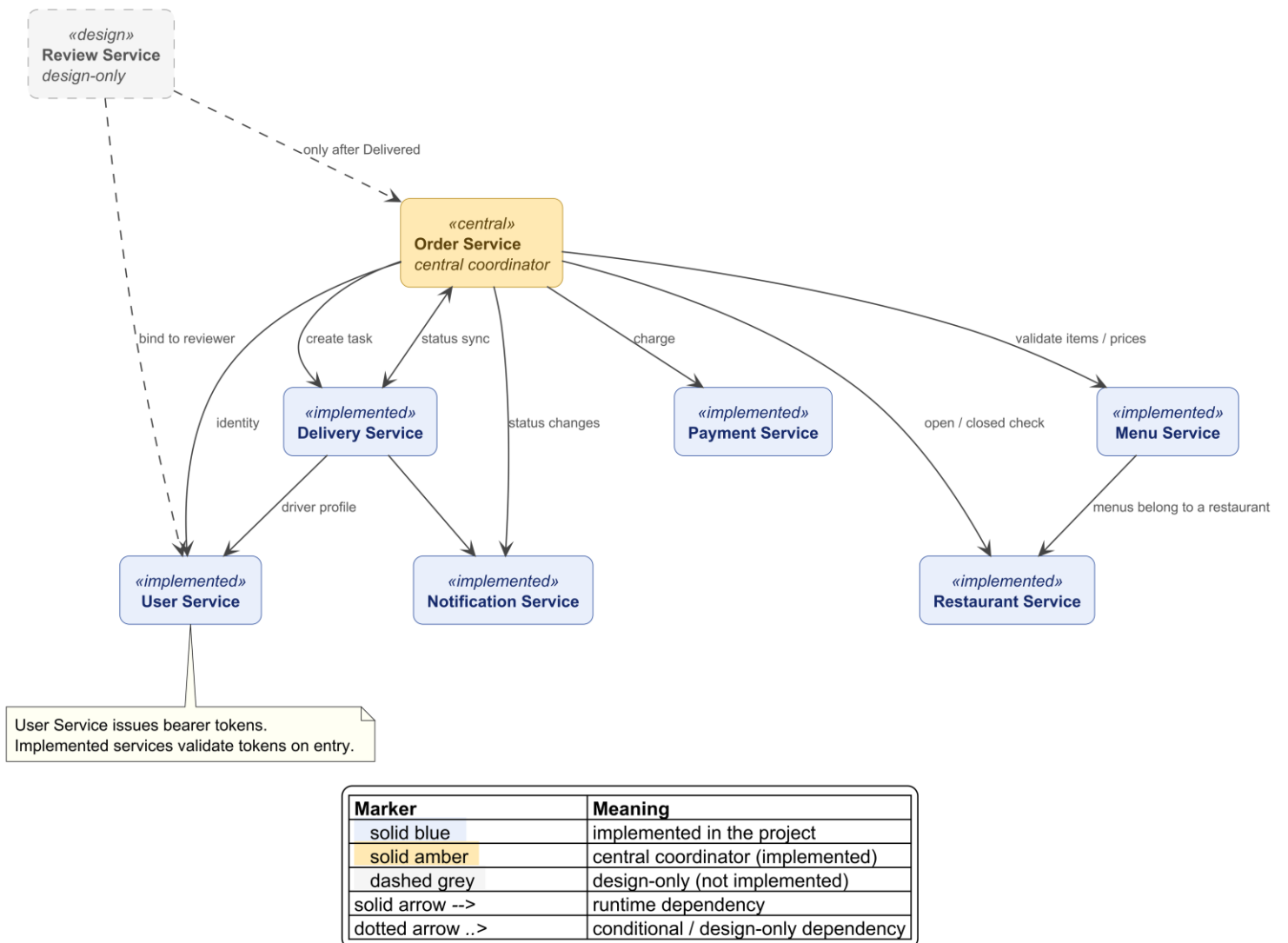


Figure 1a — Complete business architecture diagram

2.3 Implementation Architecture (technical view)

We keep the technical runtime view separate from the business architecture so infrastructure concerns do not clutter the business view. The implementation architecture shows the client entry point, the API Gateway, the implemented services, the Kafka topics, and one database per implemented business service.

This technical view is simplified to match the actual project scope:

- the client calls only the API Gateway;
- the API Gateway routes requests to the implemented REST services, including the User Service for sign-in, token issuance, and profile access;
- after authentication through the User Service, the gateway forwards the bearer token and propagates the authenticated user id to downstream services that need user context;
- each implemented microservice validates the incoming bearer token locally before processing the request;
- synchronous service-to-service REST calls carry a service token, so internal calls are also authorized;
- the Order Service performs synchronous REST calls to User, Restaurant, Menu, Payment, and Delivery during Workflow W1;
- the Delivery Service reads driver profiles from the User Service when matching drivers to deliveries;
- Payment, Delivery, and Order publish domain events to Kafka, while Order and Notification consume the relevant topics;
- we do **not** add a separate service-discovery server, because static service configuration is enough at the scale of this project.

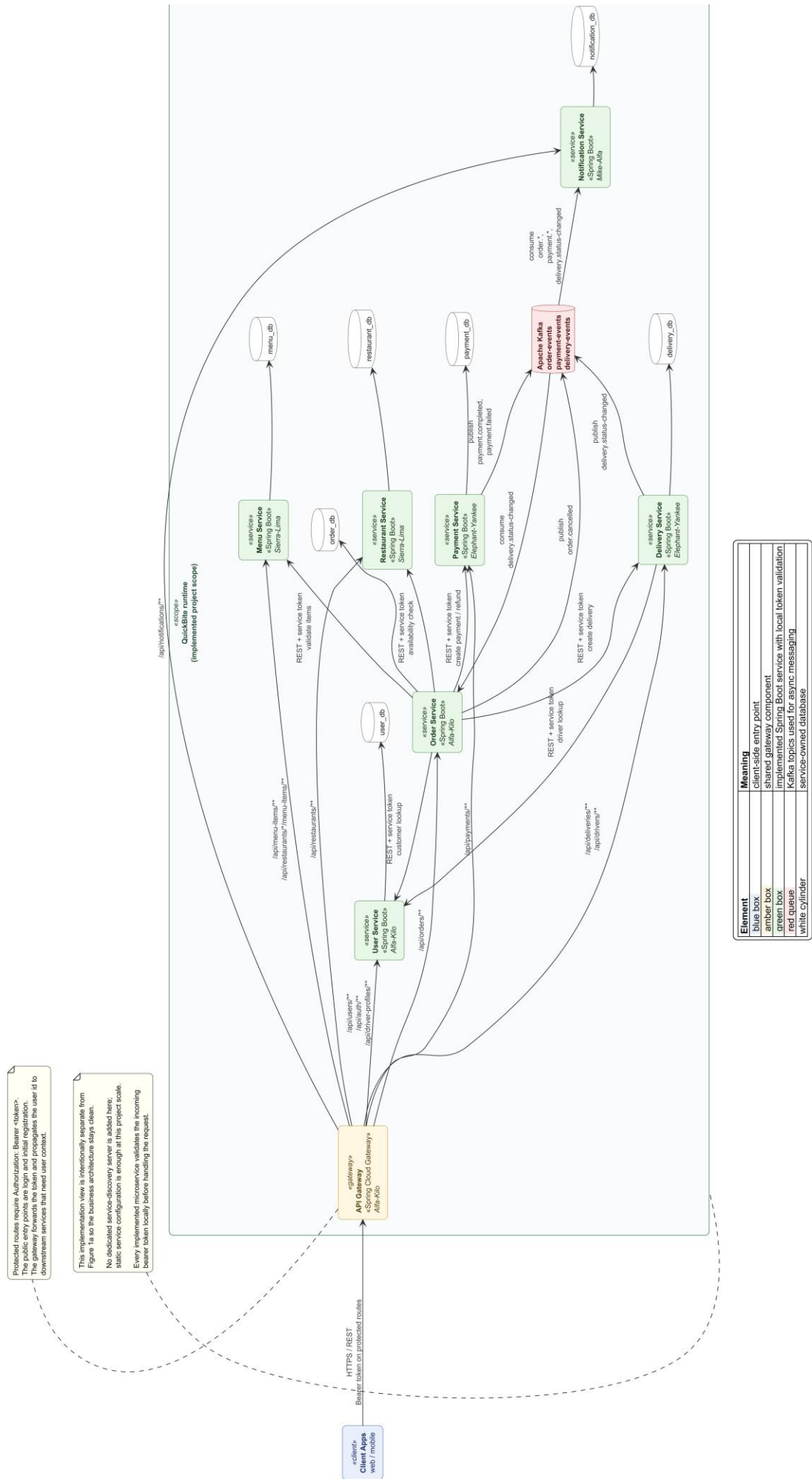


Figure 1b — Implementation architecture diagram

2.4 Subset Implemented in the Project

To keep the scope realistic, we implement only a part of the system. Each team member takes on at least two components. Sierra-Lima, Elephant-Yankee, and Mike-Alfa each keep two components, while Alfa-Kilo takes on a third component by implementing the User Service in addition to Order Service and API Gateway.

Status	Services / Components
Implemented (business services)	Order, User, Restaurant, Menu, Payment, Delivery, Notification
Implemented (shared integration/resilience)	API Gateway (Alfa-Kilo), Event Broker configuration (Mike-Alfa)
Design only (not implemented)	Review Service

Why this choice. Review Service stays in the design because post-delivery reviews are important, but not required in the first implementation slice. User Service is implemented because identity, token issuance, and profile data are used in order placement and driver lookup. Keeping API Gateway and the event broker in the implementation still lets us show both synchronous integration (gateway + REST) and asynchronous integration (event broker), while now implementing seven business services. The implemented services use the real User Service for authentication, token issuance, customer lookup, and driver-profile lookup.

For the one service that is not implemented:

- no service needs to call the Review Service at run time.

3. Service APIs

We only describe the services that will be implemented in the project. Each service keeps a compact REST API, and all paths go through the API Gateway (see §3.8). Unless stated otherwise, implemented REST endpoints require `Authorization: Bearer <token>`. The public exceptions are `POST /users` (registration) and `POST /auth/login` (sign-in). Internal service-to-service REST calls also include a bearer token issued for the calling service.

3.1 Order Service (Owner: Alfa-Kilo)

Endpoint	Method	Description
/orders	POST	Place a new order. Uses the authenticated customer identity from User Service, validates the restaurant and the menu items, starts payment, and creates the delivery task.
/orders/{id}	GET	Get the details and current status of an order.
/orders/{id}	DELETE	Cancel a pending order. Only allowed before the restaurant accepts it.
/orders/{id}/status	PATCH	Internal status change. Called by Payment and Delivery through event handlers.

/orders?customerId={id}	GET	List the orders of a given customer (order history).
/orders/{id}/items	GET	List the items of a given order.

3.2 User Service (Owner: Alfa-Kilo)

Endpoint	Method	Description
/users	POST	Register a new user account (customer, driver, restaurant owner, or admin).
/auth/login	POST	Sign in and return the authenticated user context/token used through the API Gateway.
/users/{id}	GET	Get a user profile.
/users/{id}	PUT	Update a user profile.
/users/{id}/addresses	GET	List the saved addresses of a given user.
/users/{id}/addresses	POST	Add a new saved address for a given user.
/driver-profiles?available=true	GET	List available driver profiles for delivery assignment.

POST /users and POST /auth/login are the public entry points. The remaining User Service endpoints require a bearer token. The User Service is called synchronously by Order Service for customer lookup and by Delivery Service for driver lookup.

3.3 Restaurant Service (Owner: Sierra-Lima)

Endpoint	Method	Description
/restaurants	POST	Register a new restaurant profile.
/restaurants/{id}	GET	Get a restaurant profile.
/restaurants/{id}	PUT	Update the restaurant profile (hours, location, details).
/restaurants/{id}/status	PATCH	Change the open/closed status.
/restaurants	GET	Search or list restaurants (filters: city, open).
/restaurants/{id}/availability	GET	Quick check to see if a restaurant is open. Called by Order Service during order placement.

3.4 Menu Service (Owner: Sierra-Lima)

Endpoint	Method	Description
/restaurants/{rid}/menu-items	POST	Add a new menu item to a restaurant.
/restaurants/{rid}/menu-items	GET	List the menu items of a given restaurant.
/menu-items/{id}	GET	Get a single menu item (price and availability).
/menu-items/{id}	PUT	Update a menu item.
/menu-items/{id}	DELETE	Remove a menu item.

/menu-items/validate	POST	Validate a list of {menuItemid, quantity} and return the unit prices. Called by Order Service during order placement.
----------------------	------	---

3.5 Payment Service (Owner: Elephant-Yankee)

Endpoint	Method	Description
/payments	POST	Start a payment charge for an order.
/payments/{id}	GET	Get a payment and its status.
/payments/by-order/{orderId}	GET	Get the payment that belongs to a given order.
/payments/{id}/refund	POST	Issue a refund for a cancelled or failed order.
/payments/{id}/transactions	GET	List the transactions (charge or refund) of a payment.

The Payment Service also publishes the asynchronous events *payment.completed* and *payment.failed* (see §6).

3.6 Delivery Service (Owner: Elephant-Yankee)

Endpoint	Method	Description
/deliveries	POST	Create a delivery task for a paid order.
/deliveries/{id}	GET	Get a delivery task.
/deliveries/{id}/assign	POST	Assign an available driver to a delivery task.
/deliveries/{id}/status	PATCH	Driver updates the status (Assigned → PickedUp → Delivered).
/deliveries/by-order/{orderId}	GET	Get the delivery task that belongs to a given order.
/drivers/{id}/active	GET	Get the active delivery of a given driver.

The Delivery Service also publishes the asynchronous event *delivery.status-changed* (see §6).

3.7 Notification Service (Owner: Mike-Alfa)

Endpoint	Method	Description
/notifications	GET	List the notifications of the current user.
/notifications/{id}	GET	Get a single notification.
/notifications/unread-count	GET	Get the number of unread notifications for the current user.
/notifications/{id}/read	PATCH	Mark one notification as read.
/notifications/read-all	PATCH	Mark all notifications of the current user as read.
/notifications/send	POST	Internal send endpoint. Used for testing and admin actions only.

The Notification Service is mainly event-driven. It consumes `order.*`, `payment.*`, and `delivery.status-changed` events (see §6).

3.8 API Gateway (Owner: Alfa-Kilo — shared integration component)

The API Gateway is not a REST service on its own. It is the single entry point for clients and it handles cross-cutting concerns. We use Spring Cloud Gateway.

Route prefix	Target service	Access
/api/orders/**	Order Service	Protected
/api/users/**, /api/auth/**, /api/driver-profiles/**	User Service	POST /api/users and /api/auth/login are public; the remaining routes are protected
/api/restaurants/**	Restaurant Service	Protected
/api/menu-items/**, /api/restaurants/*/menu-items/**	Menu Service	Protected
/api/payments/**	Payment Service	Protected
/api/deliveries/**, /api/drivers/**	Delivery Service	Protected
/api/notifications/**	Notification Service	Protected

The gateway also does:

- request routing and path rewriting;
- authentication filtering on protected routes;
- CORS handling in one place;
- simple rate limiting;
- forwarding the bearer token and authenticated user id from the User Service to downstream services that need user context;
- leaving token validation enabled in each downstream microservice, so the gateway is not the only enforcement point.

3.9 Messaging / Event Broker (Owner: Mike-Alfa — shared integration component)

This is also not a REST service. It provides the messaging layer used by the asynchronous workflows in §5 and §6. We use **Apache Kafka**, because it has good support in Spring Boot (through Spring for Apache Kafka) and is easy to run as one container in our demo setup. The topics, producers, and consumers are listed in §6.2.

4. Service Data Models

Each implemented service has its own database. No service reads another service's tables directly. Cross-service links are stored as ID fields (marked with *), the same rule as in Assignment 2.

4.1 Order Service DB (*Alfa-Kilo*)

Entities

- **Order** (*aggregate root*) — orderId (PK), customerId*, restaurantId*, status (Placed | Paid | Confirmed | Prepared | Ready | PickedUp | Delivered | Cancelled), placedAt, totalAmount, deliveryStreet, deliveryCity, deliveryPostalCode.
- **OrderItem** — orderItemId (PK), orderId (FK), menuItemId*, name, unitPrice, quantity.

Relationships. One Order has many OrderItem rows.

4.2 User Service DB (*Alfa-Kilo*)

- **User** (*aggregate root*) — userId (PK), email, passwordHash, fullName, phoneNumber, role (Customer | Driver | RestaurantOwner | Admin), status (Active | Suspended), createdAt.
- **Address** — addressId (PK), userId (FK), street, city, postalCode, label, isDefault.
- **DriverProfile** — userId (PK / FK to User.userId), vehicleType, licenseNumber, isAvailable.

Relationships. One User has many Address rows. A user with role Driver may also have one DriverProfile.

Security note. Access tokens are signed bearer tokens derived from authenticated user data, so we do not add a separate shared token table to the database model.

4.3 Restaurant Service DB (*Sierra-Lima*)

- **Restaurant** (*aggregate root*) — restaurantId (PK), ownerId*, name, address, city, latitude, longitude, operatingHours, isOpen.

4.4 Menu Service DB (*Sierra-Lima*)

- **MenuItem** (*aggregate root*) — menuItemId (PK), restaurantId*, name, description, priceAmount, priceCurrency, category, isAvailable.

Relationships. Logical: many MenuItem rows belong to one restaurant (restaurantId*).

4.5 Payment Service DB (*Elephant-Yankee*)

- **Payment** (*aggregate root*) — paymentId (PK), orderId*, amount, currency, status (Pending | Completed | Failed | Refunded), processedAt.
- **Transaction** — transactionId (PK), paymentId (FK), type (charge | refund), amount, occurredAt.

Relationships. One Payment has many Transaction rows.

4.6 Delivery Service DB (*Elephant-Yankee*)

- **DeliveryTask** (*aggregate root*) — deliveryId (PK), orderId*, driverId*, pickupStreet, pickupCity, dropoffStreet, dropoffCity, status (Pending | Assigned | PickedUp | Delivered), assignedAt.

4.7 Notification Service DB (Mike-Alfa)

- **Notification** (*aggregate root*) — notificationId (PK), recipientId*, channel (email | sms | push), message, sentAt, status (Queued | Sent | Failed | Read).

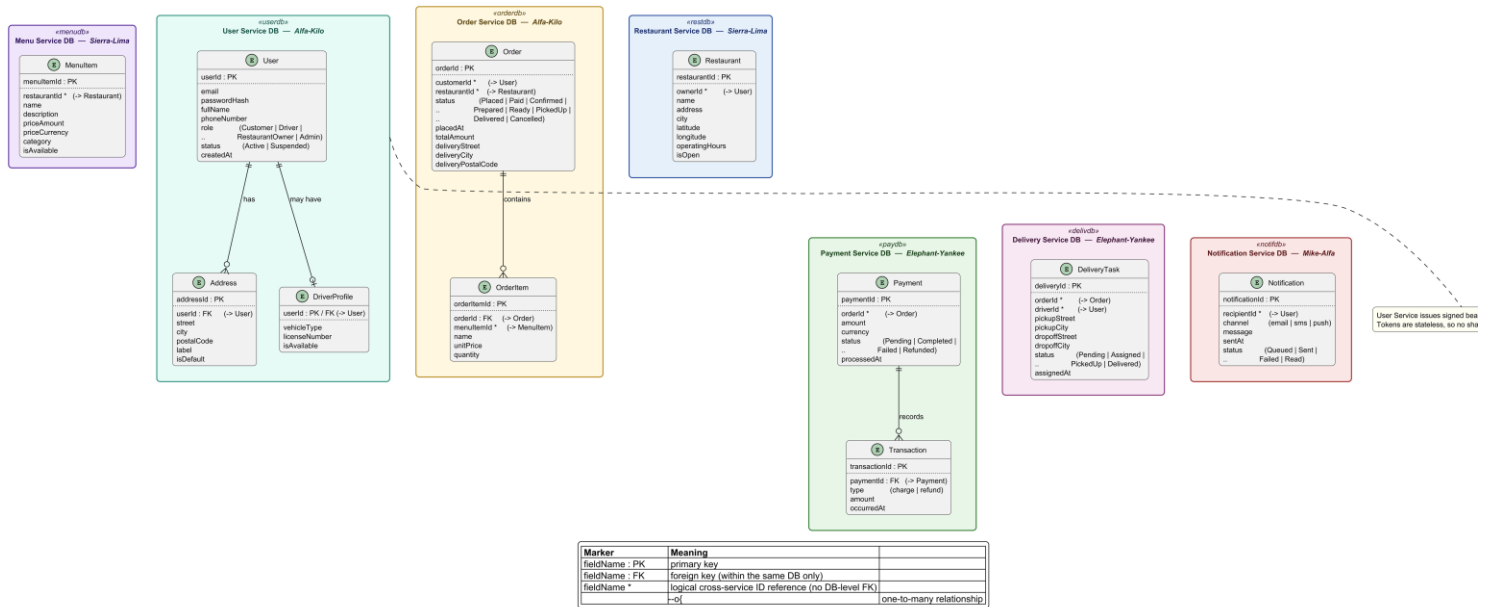


Figure 2 — ER diagram for each implemented service

5. System Workflows

This section presents the main end-to-end workflows. Workflows W1 and W2 are selected for implementation in the project. Together, they cover **both** required interaction styles:

- W1 shows synchronous REST-based coordination;
- W2 shows asynchronous event-driven integration.

W3 is an extra asynchronous workflow that goes together with W1. W4 is kept at the design level only.

5.1 W1 — Place Order (*synchronous, implemented*)

Trigger. An authenticated customer submits an order from the client app.

Precondition. The client has already signed in through the User Service and holds a bearer token.

Services involved. API Gateway, User Service, Order Service, Restaurant Service, Menu Service, Payment Service, Delivery Service.

Steps.

1. Client → API Gateway → **Order Service**: POST /orders with Authorization: Bearer <token> and {restaurantId, items[], deliveryAddress}. The gateway and the Order Service both validate the token, and the gateway forwards the authenticated customerId obtained through the User Service.
2. Order Service → **User Service**: GET /users/{customerId} to confirm that the customer exists and has an active account.
3. Order Service → **Restaurant Service**: GET /restaurants/{id}/availability to confirm that the restaurant is open.

4. Order Service → **Menu Service**: POST /menu-items/validate with the list of items. This checks that each item exists and is available, returns the unit prices, and lets the Order Service compute totalAmount.
5. Order Service saves the order with status = Placed in its own database (Order and OrderItem tables).
6. Order Service → **Payment Service**: POST /payments with {orderId, amount}. The charge runs synchronously.
7. If the payment result is Completed, Order Service → **Delivery Service**: POST /deliveries with {orderId, pickupAddress, dropoffAddress}. The order status is then updated to Paid, and then to Confirmed.
8. Order Service returns 201 Created to the client, with the order id and the first status.

The REST calls from Order Service to User, Restaurant, Menu, Payment, and Delivery also carry a service bearer token, and each called service validates it before processing the request.

Tables updated. orders, order_items (Order DB); payments, transactions (Payment DB); delivery_tasks (Delivery DB).

Failure handling.

- If step 2 fails, the Order Service returns 401 or 404.
- If step 1 has a missing, expired, or invalid token, the gateway or the Order Service returns 401 or 403.
- If step 4 fails, the Order Service returns 422.
- If step 6 fails, the order is set to Cancelled and no delivery task is created.
- If step 7 fails, the Order Service sets the order to Cancelled, calls POST /payments/{id}/refund, and may publish order.cancelled for notification.

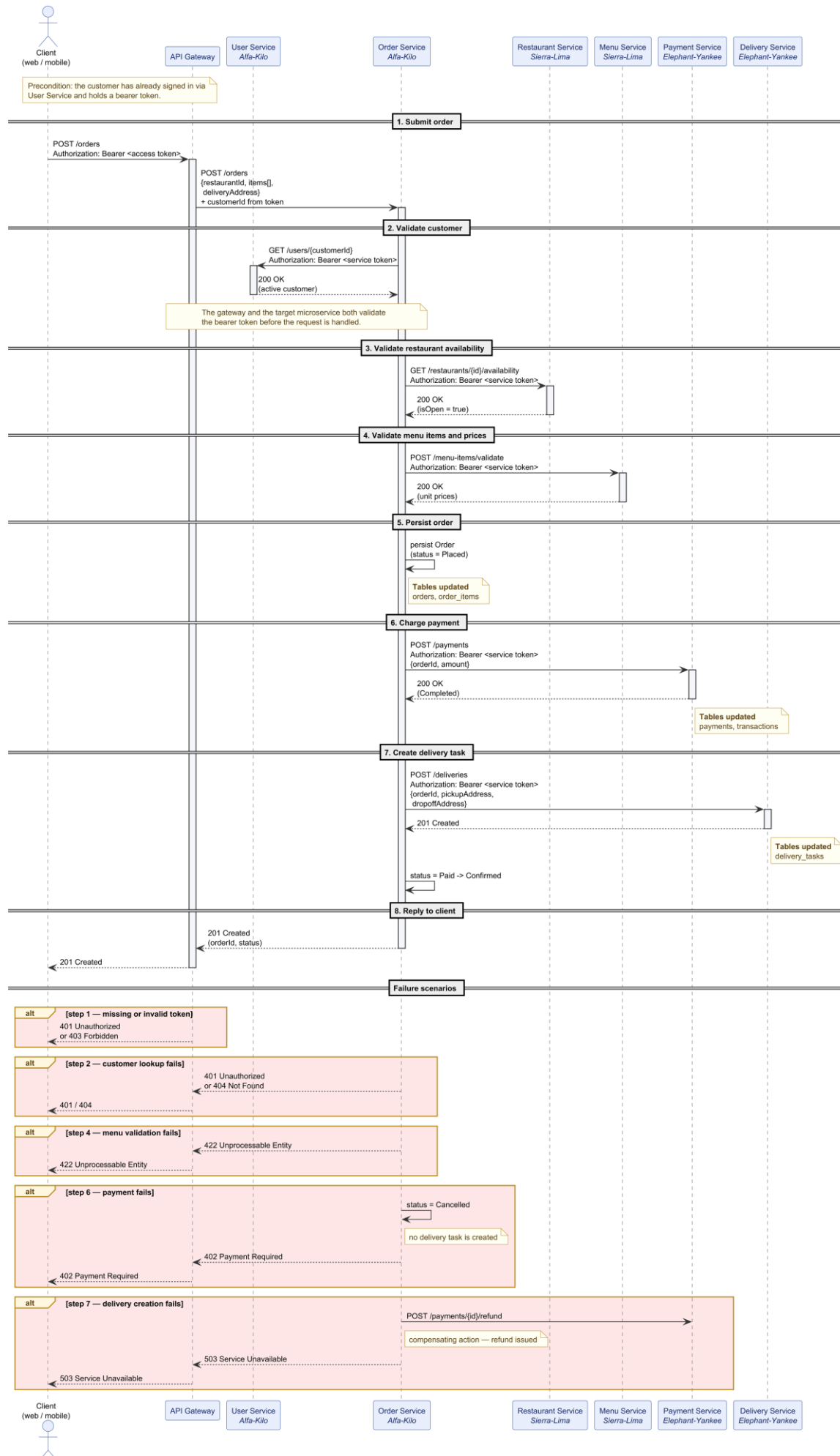


Figure 3 — Sequence diagram of Workflow W1

5.2 W2 — Delivery Progress & Notifications (*asynchronous, implemented*)

Trigger. An authenticated driver updates the delivery status in the Delivery Service.

Services involved. API Gateway, Delivery Service (producer), Order Service (consumer), Notification Service (consumer), Event Broker.

Steps.

1. Driver client → API Gateway → **Delivery Service**: PATCH /deliveries/{id}/status with Authorization: Bearer <token> and the new value (PickedUp or Delivered).
2. The API Gateway and the Delivery Service validate the token and the driver role.
3. Delivery Service saves the new status in its own database.
4. Delivery Service **publishes** the event delivery.status-changed to the topic delivery-events with the payload {deliveryId, orderId, status, occurredAt}.
5. **Order Service** (consumer) updates the matching order status (PickedUp → PickedUp; Delivered → Delivered) in its own database.
6. **Notification Service** (consumer) creates a user-facing notification for the customer and the driver, saves it, and records the send attempt.

Tables updated. delivery_tasks (Delivery DB); orders (Order DB); notifications (Notification DB).

The event flow is asynchronous after step 3, but the REST endpoint that triggers it is still token-protected.

5.3 W3 — Payment Result → Customer Notification (*asynchronous, implemented as complement to W1*)

After step 6 of W1, the Payment Service also publishes payment.completed or payment.failed to the payment-events topic. The Notification Service consumes these events and informs the customer whether the payment was accepted or rejected. This keeps notification logic out of the synchronous order-placement path, while Order Service handles payment success or failure directly in W1.

5.4 W4 — Submit Review (*design only, not implemented*)

Included for completeness. A customer submits a review for a delivered order. The Review Service checks with the Order Service before saving the review. This workflow is not implemented, because the Review Service is design-only.

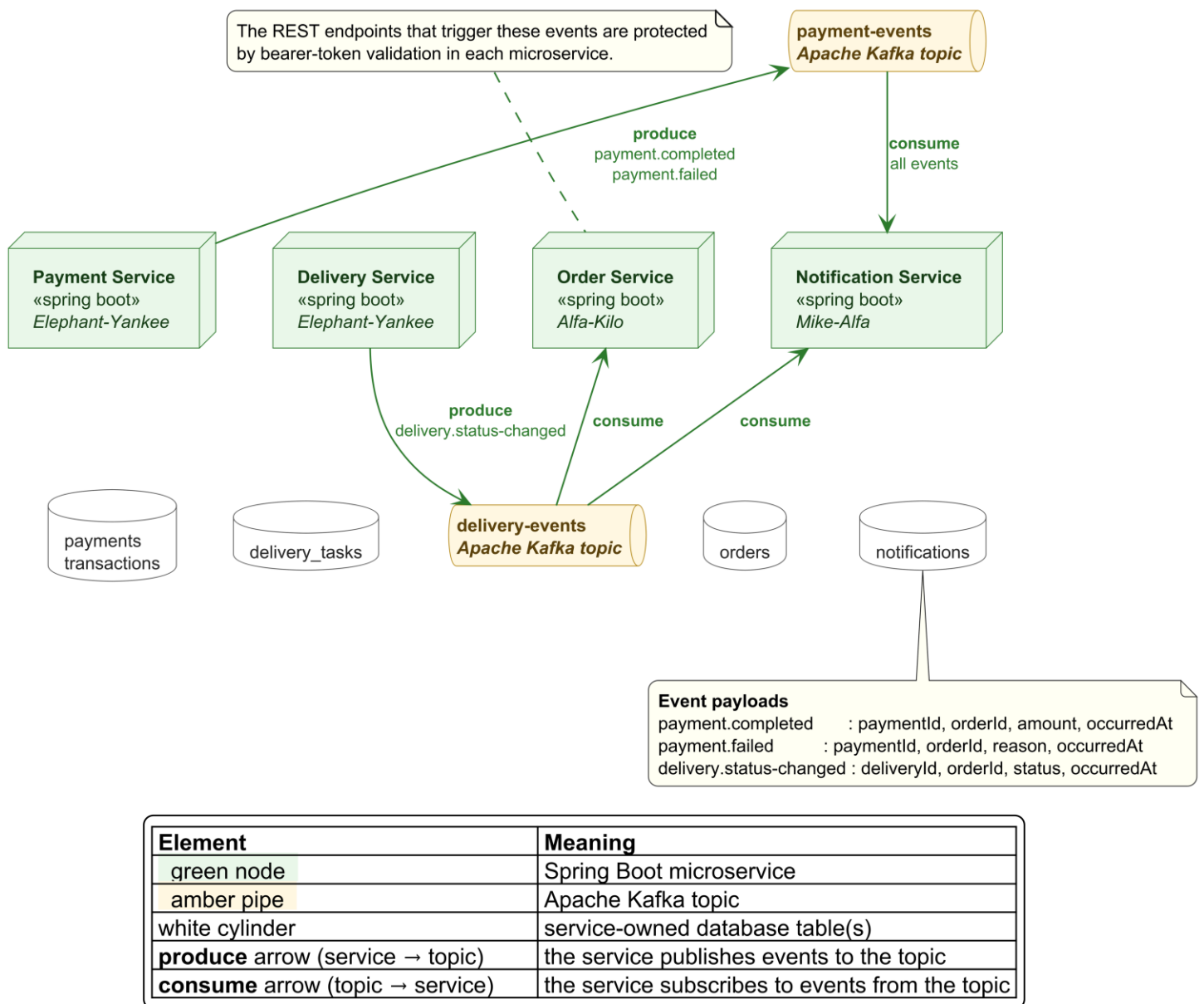


Figure 4 — Event-flow diagram for workflows W2 and W3

6. Integration Mechanisms

6.1 Synchronous integration (REST)

- **Transport.** HTTP with JSON bodies, through the **API Gateway** (Spring Cloud Gateway). Protected requests carry Authorization: Bearer <token>, and each Spring Boot service validates the token locally before processing the call. Service-to-service calls inside the system are direct REST calls and carry a service token. Static configuration is enough at the scale of this project, so we do not add a separate service-discovery server.
- **Public entry points.** POST /auth/login and POST /users are public so a new or returning user can obtain a token. The remaining implemented REST endpoints are protected.
- **Used in.** W1 (Order ↔ User, Restaurant, Menu, Payment, Delivery), login/profile access through the gateway, delivery creation, and menu validation.

- **Error handling.** HTTP status codes are mapped to domain errors. Missing or invalid tokens lead to 401 or 403, and the Order Service runs compensating actions (for example, a refund) when a downstream call fails.

6.2 Asynchronous integration (events)

An **event broker** carries the domain events. We use **Apache Kafka**, and the broker configuration is owned by **Mike-Alfa** as the shared integration component.

Topic / Exchange	Event	Producer	Consumer(s)	Payload data
payment-events	payment.completed	Payment Service	Notification Service	paymentId, orderId, amount, occurredAt
payment-events	payment.failed	Payment Service	Notification Service	paymentId, orderId, reason, occurredAt
delivery-events	delivery.status-changed	Delivery Service	Order Service, Notification Service	deliveryId, orderId, status, occurredAt
order-events	order.cancelled	Order Service	Notification Service	orderId, reason, occurredAt

Refund execution remains a direct REST compensation between Order Service and Payment Service; the order.cancelled event is published only for downstream notification.

Guarantees.

- At-least-once delivery.
- Consumers are idempotent: the event id is used as the key, so a duplicate event is handled only once.
- Failed handlers are routed to a dead-letter queue.

6.3 Cross-cutting

- **Security layer** — token-based authorization on every implemented microservice. The User Service authenticates users and issues access tokens, the API Gateway checks protected routes and forwards the token, and each downstream service validates it again locally.
- **API Gateway** (Alfa-Kilo) — one entry point for clients, request routing, CORS handling, basic rate limiting, and propagation of the authenticated user id obtained through the User Service.
- **Event Broker configuration** (Mike-Alfa) — declares topics and queues, sets the dead-letter policy, and defines a shared event envelope (id, type, occurredAt, payload).

7. Final Implementation Responsibilities

Each team member implements at least two components. Sierra-Lima, Elephant-Yankee, and Mike-Alfa each keep two components. Alfa-Kilo implements a third component by taking on the User Service in addition to Order Service and API Gateway. All four team members still keep at least one business service that has a non-trivial REST API, owns its own database, and takes part in at least one implemented workflow.

Team member	Component 1 (business service)	Component 2	Component 3 (if any)	Notes
Alfa-Kilo	Order Service — REST API (§3.1), DB (§4.1), main coordinator of W1, consumer in W2	User Service — REST API (§3.2), DB (§4.2), provides authentication, token issuance, and customer/driver identity data for W1 and delivery matching	API Gateway (shared integration component)	Owns identity, token issuance, gateway routing, and entry-point security in addition to the order core.
Sierra-Lima	Restaurant Service — REST API (§3.3), DB (§4.3), takes part in W1	Menu Service — REST API (§3.4), DB (§4.4), takes part in W1	-	Both business services are implemented.
Elephant-Yankee	Payment Service — REST API (§3.5), DB (§4.5), takes part in W1 and produces payment-events	Delivery Service — REST API (§3.6), DB (§4.6), takes part in W1 and produces delivery-events (W2)	-	Both business services are implemented.
Mike-Alfa	Notification Service — REST API (§3.7), DB (§4.7), consumer in W2 and W3	Event Broker configuration (shared integration component, replaces Review Service)	-	Review Service stays design-only.

Constraint checks.

- Every team member implements at least two components. ✓
- Alfa-Kilo implements a third component (User Service) in addition to Order Service and API Gateway. ✓
- Mike-Alfa uses the replace-one-service option for Review Service, while the other team members keep their original business-service scope. ✓
- Every team member who owns a shared component still has at least one substantial business service with a REST API, its own data, inter-service interactions, and participation in implemented workflows. ✓
- Every implemented microservice exposes token-protected REST access, with registration and login as the public entry points. ✓
- The selected workflows cover both synchronous (W1) and asynchronous (W2, W3) interactions. ✓
- At least seven business services are implemented, so the system stays realistic and coherent. ✓

8. Summary

The QuickBite implementation plan covers seven business microservices plus two shared integration components. They are driven by two end-to-end workflows, which together show both synchronous REST-based coordination and asynchronous event-driven integration. All implemented REST services are protected by bearer-token authorization, with the User Service handling authentication and token issuance. The work is clearly split between the four team members, with Alfa-Kilo taking on the extra User Service alongside Order Service and API Gateway.